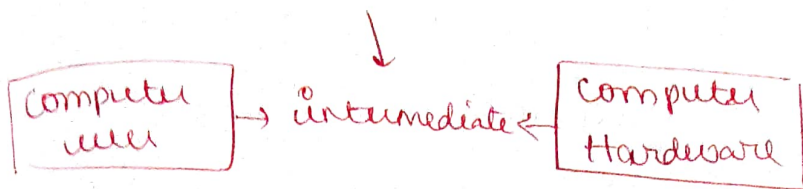


OS



Bootstrap program: -- initial program that runs when computer is powered up (ROM)
- locates and loads memory the OS kernel.

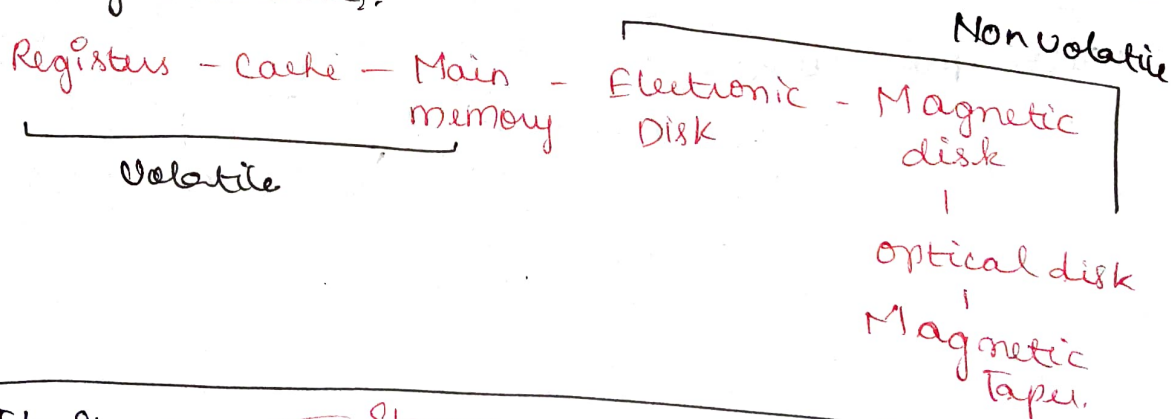
Interrupt: - the occurrence of an event is usually signalled by an Interrupt.

System call (Monitor call): - Software may trigger an interrupt by system call.

=> when CPU is interrupted, stores current process & immediately transfers execution to fixed location

Service routine of interrupt + (ISR)

Storage Structures.



I/O Structure — **Storage**
(large portion of OS)
↳ connected through buses.

device controller: -- maintains
↳ local buffer storage
↳ Set of special purpose registers.

device driver: understands controller & presents to OS

IO working (I/O)

- the device driver loads appropriate reg.
- in device controller, examines driver.
- Sends to local buffer.
- then device buffer informs driver via interrupt
- device driver then returns control to OS

for bulk data

→ Direct memory access (DMA)

- ↓
- after buffer, the device controller transfers data directly to memory (no CPU)
- per block = one interrupt

Types

1) Single processor system:-

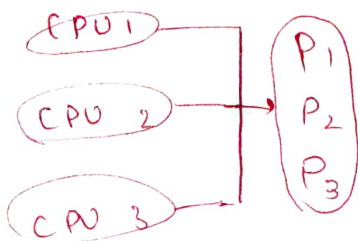
- One CPU (general purpose)
- other special processor for specific tasks (micro)

2) Multi processors:-

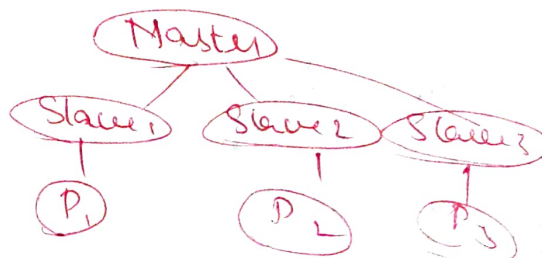
- Parallel / tightly coupled
- has 2 or more in close communication, sharing bus (clock, memory).

- ⇒ Increased throughput
- ⇒ Economy of Scale
- ⇒ Increase reliability

Symmetric MP



Asymmetric MP



3) Cluster Systems -

- 2 or more individual system coupled together
- high availability
- structure / asymmetric

Multiprogramming vs Multitasking (Time sharing)

i) Multiprogramming:-

- a single user cant keep CPU or I/O busy all time.
- Multiprogramming increases CPU utilisation by organising data so CPU busy all the time.

ii) Multitasking:- [Time sharing]

- CPU executes Multiple jobs by switching among them
- Switches so frequently that user can interact with each program while its running.
- Direct communication B/w user & system.
- Many user can share the system. [CPU scheduling]

* A program loaded into memory & executing is called a Process.

OS Services

- 1) User Interface (CLI / GUI)
- 2) Program execution
- 3) I/O operations
- 4) File System manipulation
- 5) communications (B/w processes)
- 6) Error detection (Printer no paper)
- 7) Resource allocation
- 8) Accounting (for user logging)
- 9) Protection & Security.

CLI

↳ On systems with multiple interpreters to choose, the interpreters are known as shells.

Kernel: - (0)

- The first program after the bootloader, it remains in RAM for entire duration the computer is powered on.
- executes with full hardware access.

- o Process management
- o Memory management
- o Interrupt handling
- o Device Management
- o File System management

Structures of OS

Monolithic Kernel:-

- Everything packed into single binary that runs inside Ring 0.

Microkernel

- Strip kernel down to bare min needed to coordinate
- Basic IPC & low-level CPU Scheduling

Hybrid kernel

- Hybrid (Keeps basic IPC & device drivers in Ring 0)
- Faster but bit risky

Exo Kernel:- (Basically gives users

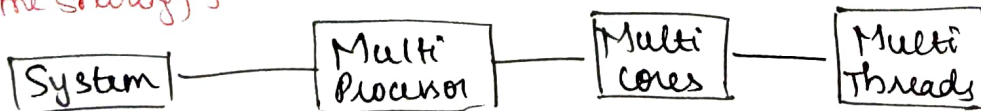
- the kernel becomes very small and mainly does resource protection + resource allocation.



OVERVIEW

Multiprocessing:- When CPU idle, Switch to another app.

Multitasking / (Time Sharing):- Avoids starvation:- time sliced ✓



Multi Processors:- each processor can execute an app, independent of others

Synchronisation:-

- occurs when 2 or more apps attempt to read & write same shared resource at same time
- OS associates shared resource with lock

OS & Isolation:

1) OS Isolation:

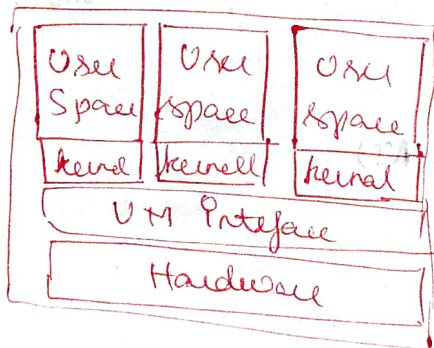
- Ring 0 :- Kernel (Privileged) - can access any hardware memory
- Ring 3 :- User mode - Restricted.
- Mode bit :- 0 = Kernel / 1 = User

2) Program Isolation:-

- Use virtual memory to ensure programs are isolated.
- Set permission :- Execute; Read; Read write.

System call provides interface.	Procedure call
TRAP instruction ✓ User to Kernel ✓ Jumps to fixed addr ✓	- CALL instruction - Stays in User - Jumps to relocatable address.

Virtual Machine



3 Type of addresses:-

- 1) Memory addresses:- RAM ($0 - 2^{32} - 1$) (Load/Store)
- 2) I/O port addresses:- Identifies an I/O port ($0 - 2^{16} - 1$)
- 3) Memory mapped I/O:- device registers using memory address space (GPIO).

Legacy 1 MB Memory map (The 8088 Heritage)

- Intel 8088 has 20 address pin
- Space :- $2^{20} - 1 = 1 \text{ MB}$

640 KB :- Free physical RAM ✓

640 - 1 MB :- Video memory.

Real Mode:-

- address are 16bit
- but 1MB = 2^{20} bytes.

Protected Mode :-

- 32 bytes:- $2^{32} = 4 \text{ GB}$.
- introduced Ring 0 & Ring 3.

Physical address = (Segment $\times 16$) + Offset

A20 Gate:-

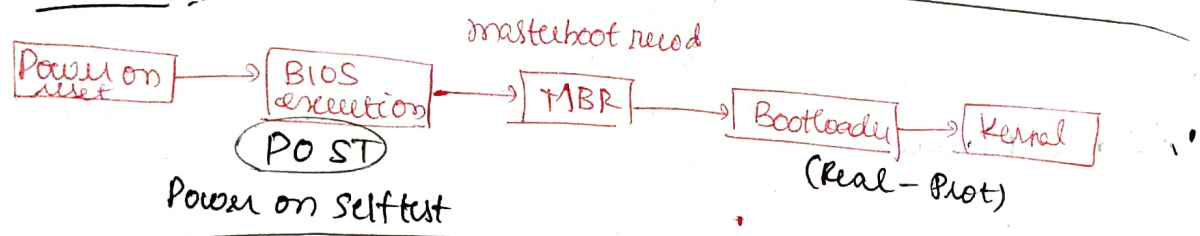
(The address 20 address)
A20 bit can't exist,
the address wrapped to
0x00000 [0x100000].

Seg = FFFF
Offset = 0010
 $FFFF \times 16 = FFFF0$
 $+ 0010 = FFFF2$
 $FFFF2 - 100000 = 00000$

Note:-

- A20:- disabled - wrap around @ 1MB
- enabled - can access

PC-boot :-



Multiprocessing boot:- (BSP vs AP)

1) Bootstrap processor (BSP):-

- During Startup, BIOS → one single core as BSP
- All other are application processor (halted)

BIOS, MBR,
Bootloader

2) Waking up APs (Startothers (IPI)) :-

- only BSP finishes initialising the basic kernel, memory, it wakes up remaining
- Mech:- BSP sends special signal called Startup IPI via local APIC controller to AP core (Interrupts)

Types of system calls:-

- 1) process control (Manage process) - fork(), exec(), ...
- 2) File Manipulation
- 3) Device Manipulation (Printer, disk)
- 4) Information maintenance (Time/date)
- 5) Communication (inter process communication)
- 6) Memory management (malloc(), ...)

Structure of OS

- 1) Simple Structure: - app can directly read/write Hardware
- 2) Monolithic Structure: - all OS services & drivers are compiled into one giant static binary ring 0.
- 3) ~~Hybrid~~ layered structure: - Stacked layers (0 - hardware; N - UI)
- 4) Microkernel: - IPC + CPU scheduling (Ring 0 + Ring 3)
- 5) Modular Structure: - A small core kernel in ring 0 that dynamically loads & unloads Modules.

Memory Management } later
Interrupts & Syscalls }

Process & Context Switching.

Process: - a program in active execution with own Virtual space, CPU reg etc.

⇒ 2 stacks per process:-

- i) User stack (ring 3): - Normal func, local variables
- ii) Kernel stack (ring 0): - 4KB stack inside kernel for each process.

program	process
Code + Static + Global data	Dynamic instantiation → of Code + data + Heap + Stack + process stack
→ Can create several processes	→ Unique isolated entity

Advantage of Virtual address map:-

- Isolation
- Relocatable
- Size

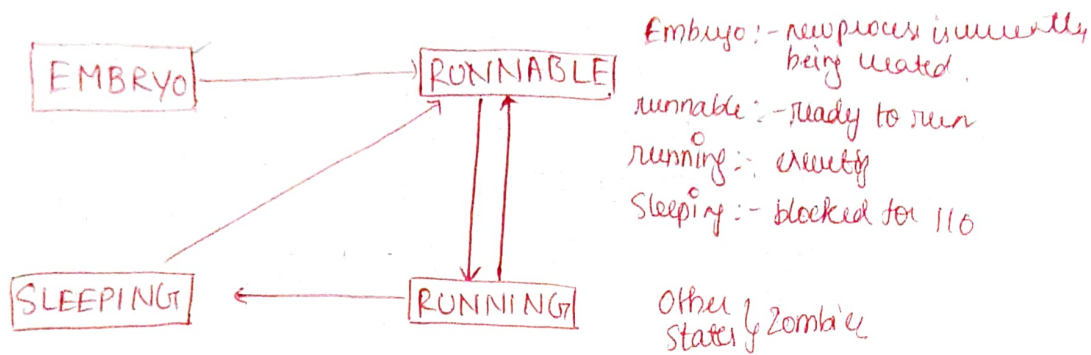
PCB (Process control block): -

⇒ a data structure maintained by the OS for every process.

- Process identification: PID, PPID
- Process state
- CPU context: PC, SP
- memory management info
- I/O & accounting info

Into categories
Stored in PCB ✓

Process state



Definitions:-

- a) Physical memory: - actual hardware RAM
- b) Page frame: - fixed subdivision
- c) Virtual memory: - high-level abstraction presenting logic, isolated address space to program
- d) Page: - fixed size subdivisions of that virtual memory
- e) Paging: - Mechanism: - maps virtual pages to physical page frames

Fork(): - process creation

↳ creates an exact duplicate of the calling process (clones space, file descriptors, registers)

in parent: - `fork()` returns child PID (> 0)

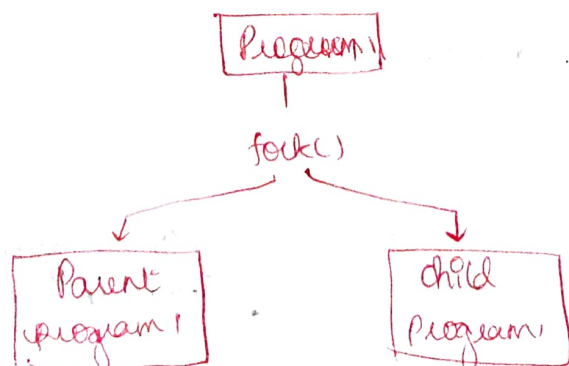
in child: - `fork()` returns 0

on failure: - returns -1

⇒ parent & child will have different copies

```
int pid;
pid = fork();
if (pid > 0) {
    pid = wait();
}
else {
    exit(0);
}
```

⇒ Copy on write: - technique used after `fork()` where parent & child share same physical memory & page copied only when one process attempts to change it



exec() :- replace the current process memory space
o exec() doesn't create new process, The PID & others are identical.

o doesn't return anything (-1 on error)

Ex:- child replacement

```
pid = fork()
if (pid == 0)
    execl("./hello", "hello", NULL);
else
    wait(NULL)
```

exec(
execp(
execv(
execvp(
)
)
)
)

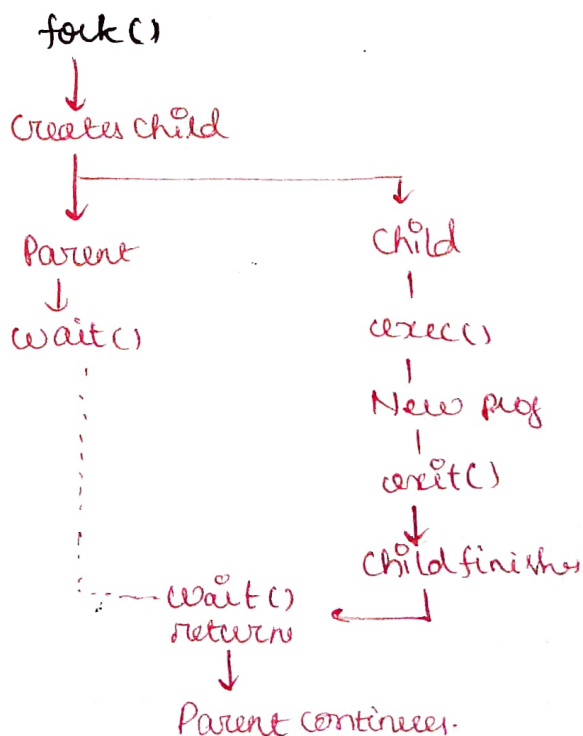
exit() :- Process termination

- closes all open files ✓
- wakes up parent sleeping in wait()
- sets state to ZOMBIE

wait() :-

- suspends parent execution until one of its children terminates
- returns PID of child that terminated ✓

Flowchart :-



Zombie Process

child
⇒ when a process terminates it becomes a zombie until its parent calls `wait()` to collect its exit information.

Child → `exit()` → ZOMBIE → Parent calls `wait()` → OS collects → zombie disappears.

Reaping process

Ex:-

```
int pid = fork()
```

```
if (pid == 0)
```

```
    printf("Child\n")
```

```
    exit(0)
```

```
else
```

```
    sleep(10)
```

```
    wait()
```

for this 10 seconds, the child process is called zombie

⊗ if parent never calls `wait()` and keeps running without reaping many zombie entries can accumulate & consume process table resource.

Orphan Process

⇒ when parent terminates before the child.

⇒ Now child lost its original parent, Orphan process

↳ The OS adopts a new parent (special system process)
UNIX/LINUX → `init(PID1)` [Systemd]

1) Unintentional:-

```
int pid = fork()
```

```
if (pid > 0)
```

```
    exit(0)
```

```
else
```

```
    sleep(10)
```

```
    printf("Child running")
```

it was not deliberately part of program

2) Intentional:-

(Daemon)

↳ to create a background process that should continue independently of original parent.

Note:-

Zombie :- child terminated, parent has not called `wait()`

Orphan :- child running, parent terminated

Note:-

- o Single fork() $\rightarrow$ 2 process
- o unconditional n fork() produce 2^n process.
fork()
fork()
printf() $\rightarrow$ printed 2 times.

context switching in xv6. (visit) - Next page

- 1 the mechanism of suspending the current process in CPU core, saving its execution context & running saved execution context of another.

Threads (at end)

CPU Scheduling! - ★★★★★

process scheduling! - Multiprogramming $\rightarrow$ Some process running all the time to max CPU utilisation.

Time sharing $\rightarrow$ Switch CPU among process so quick.

$\rightarrow$ To meet this, the process scheduler selects an available process for execution on CPU

Scheduling Queue

A) Job Queue:- As processes enter, they are put into job queue which consists all processes

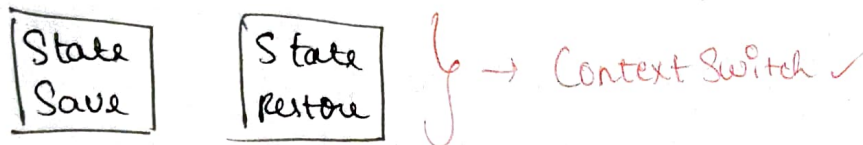
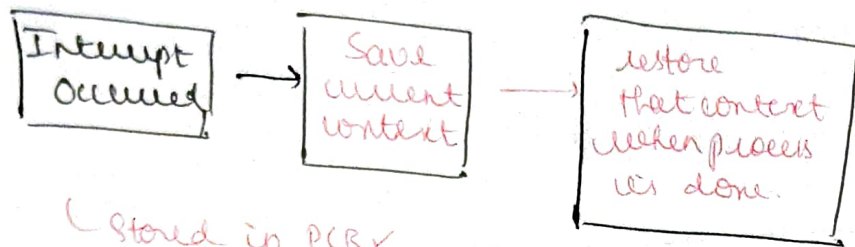
B) Ready Queue:- The process in main memory & are ready are here (waiting for CPU)

C) Device I/O Queue:- If a process needs I/O, it leaves ready queue to wait in I/O Device Queue.

D) Suspended / Swap-related Queue:- when process temporarily removed from memory.

Context Switch (Overhead)

- Interrupts cause OS to change CPU from current task
- They run frequently



→ overhead: CPU time spent saving & restoring.

CPU Scheduling: - [Basis of Multiprogrammed sys]

⇒ When one process has to wait, OS takes CPU from it and gives to another process. CPU Scheduling determines how to give it to process.

CPU & IO burst:

- process alternate B/w these 2 states
- time process is being executed in CPU (running state)
- CPU is waiting for I/O. (waiting state)

Types: - Preemptive & Non Preemptive.

- Job Scheduler
- a) Long term Scheduler: - processes from job queue to ready queue
- b) Short term Scheduler: - processes from ready queue to CPU
- c) Medium term Scheduler: - temporarily remove process from main memory later being d/ swapped

CPU Scheduler (STS): - This selects a process from the processes in memory that are ready to execute & allocates CPU to it // [ready queue]

Dispatcher: - Module that gives control of CPU to process selected by STS. The time taken to stop one and start another is dispatch latency.

CPU scheduling may take place under:-

- ① process:-
- ① running state to waiting state (I/O queue)
 - ② running state to ready state (interrupt)
 - ③ waiting state to ready state (I/O completes)
 - ④ Terminate

1, 4 $\rightarrow$ no scheduling, A new process must be selected

Preemptive:- The OS can forcibly remove the CPU from running & give it to another

Non-Preemptive:- Once a process is allocated CPU, it keeps until it voluntarily releases it.

Note:- $\star$

$\rightarrow$ When Scheduling takes place only under 1 & 4, we say that scheme is nonpreemptive or cooperative otherwise its preemptive.

(See Next Page.)

Scheduling Criteria:-

40% 90%

a) CPU utilisation:- keep CPU busy as possible (0-100%)

b) Throughput:- Measure of work:- no. of processes completed per unit time.

$$(TAT) = CT - AT$$

c) Turn-around time:- for a process, how much time it takes to complete on CPU
(Completion time - Arrival time) (waiting + ready queue + executing + I/O)

d) waiting time:- waiting time is sum of periods spent in ready queue [I/O or waiting for CPU]

$$WT = TAT - BT$$

Burst time:- CPU execution time by process

e) response time:- Time from process arrival until it gets CPU for first time

$$RT = \left[\frac{F \times AT \times CPU}{ST} \right] \cdot AT$$

$\star$ TAT is limited by speed of output device $\checkmark$ (RT needed)

Burst time (BT)

CPU burst:- amount of CPU time a process uses.

I/O burst:- amount of time process spends on I/O.

Arrival time:- (AT):- Time at which process enters ready queue.

Completion time (CT):- Time at which process finishes execution.

Turn Around time (TAT):- Total time spent by process in system.

$$TAT = CT - AT$$

Waiting time:- Time a process spends waiting in ready queue.

$$WT = TAT - BT$$

Response time:- Time from process arrival until it gets CPU for first time.

CPU Utilization:- % of CPU is busy

$$CPU \text{ util} = \frac{CPU \text{ busy time}}{Total \text{ time}} \times 100$$

Throughput:- No. of process completed per unit time

$$Throughput = \frac{No. \text{ of completed process}}{Total \text{ time}}$$

Aim for:-

- Min waiting time
- Max CPU utilisation
- Max Throughput
- Minimize response time
- Fairness

Scheduling algorithms

1) First come, First Served (FCFS)

→ the process that request the CPU first is allocated first
 implementation is done in FIFO.

Non-preemptive

Avg WT is long

Ex:-

Process	Burst time
P ₁	24
P ₂	3
P ₃	3

P₁ → P₂ → P₃ :-

0 → P₁ → 24 → P₂ → 27 → P₃ → 30

WT(P₁) = 0
 WT(P₂) = 24
 WT(P₃) = 27

AWT = $\frac{0 + 24 + 27}{3} = 17 \text{ ms}$

P₂ → P₃ → P₁ :-

0 → P₂ → 3 → P₃ → 6 → P₁ → 30

WT(P₂) = 0
 WT(P₃) = 3
 WT(P₁) = 6

AWT = $\frac{0 + 3 + 6}{3} = 3 \text{ ms}$

⇒ Troublesome for time-sharing system. (CPU share @ Interval)

Convey effect:- if processes with higher burst time arrived before then smaller processes have to wait for long time.

Q) :- do FCFS

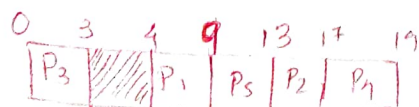
Avg wait time

& Avg Turn around.

Grant chart

Process	Arrival time	Burst time	CT	TAT	WT
P ₁	4	5	9	5	0
P ₂	6	4	17	11	7
P ₃	0	3	3	3	0
P ₄	6	2	19	13	11
P ₅	5	4	13	8	4

TAT = CT - AT
 WT = TAT - BT



ATAT = 8 units
 AWT = 4.4 units

$\frac{40}{5}$ $\frac{22}{5}$

Q)

Process	AT	BT
P ₁	0	3
P ₂	1	2
P ₃	2	1
P ₄	3	4
P ₅	4	5
P ₆	5	2

unit of overhead in scheduling. Find efficiency

Grant's chart

0	1	4	5	7	9	10	14	15	20	21	23
8	P ₁	8	P ₂	8	P ₃	8	P ₄	8	P ₅	8	P ₆

Time waited = 6 units

Useful = 17 units

Total time = 23 units

$$\eta = \frac{17}{23} = 0.7391 = 73.91\%$$

Shortest next CPU Burst alg.

2) Shortest Job first (SJF) Scheduling.

This process associates with each process the length of "Process's next CPU burst"

* CPU available → assigned to process that has the smallest next CPU burst

* It tied FCFS ✓

Preemption
(SRTF)

Non preemption
(SJF)

Shortest remaining
time first

Q)

PID	BT
P ₁	6
P ₂	8
P ₃	7
P ₄	3

Grant's chart

0	3	9	16	24
P ₄	P ₁	P ₃	P ₂	

$$w(P_1) = 3$$

$$w(P_2) = 16$$

$$w(P_3) = 9$$

$$w(P_4) = 0$$

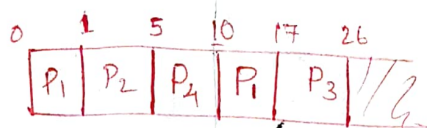
$$AWT = 7ms$$

→ Pumptive CSRTF)

PID	AT	BT
P ₁	0	8 → 7
P ₂	1	4 ✓
P ₃	2	9
P ₄	3	5

No process done at 0ms other than P₁ ✓

Gantt's chart



$$\text{waiting time} = \text{Total waiting time} - \text{No. of units Process executed} - \text{Arrival Time}$$

$$W(P_1) = (10 - 1 - 0) = 9$$

$$W(P_2) = (1 - 0 - 1) = 0$$

$$W(P_3) = (17 - 0 - 2) = 15$$

$$W(P_4) = (5 - 0 - 3) = 2$$

6.5 ms

Disadvantages:-

- the length of next CPU request can't find
- Can't do in short-term CPU scheduling

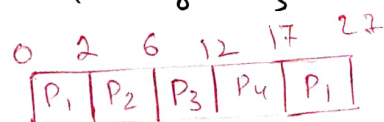
→ approx SJF

→ Predict (next CPU burst will be similar in length to previous one)

Q1) PID AT BT AWT = ?

PID	AT	BT
P ₁	0	12 10
P ₂	2	4
P ₃	3	6
P ₄	8	5

Gantt's chart



$$W(P_1) = [17 - 2 - 0] = 15$$

$$W(P_2) = [2 - 2] = 0$$

$$W(P_3) = [6 - 3] = 3$$

$$W(P_4) = [12 - 8] = 4$$

$$\frac{22}{4} = 5.5 \text{ ms}$$

Q)

PID	AT	BT
P ₁	0	16
P ₂	3	6
P ₃	7	1
P ₄	8	3

 Preemptive SRTF.

ATOT = ?

0 3 7 8 10 13 20

P ₁	P ₂	P ₃	P ₂	P ₄	P ₁
----------------	----------------	----------------	----------------	----------------	----------------

TOT(P₁) = 20 - 0 = 20ms

TOT(P₂) = 10 - 3 = 7ms

TOT(P₃) = 8 - 7 = 1ms

TOT(P₄) = 13 - 8 = 5ms

avg = $\frac{34}{4} = 8.5ms$

Note:- LTF (Longest Job first)

↳ Non-preemptive

↳ Preemptive (LRTF) ✓

4) Priority Scheduling - each will have priority & CPU given first to process with higher priority (low number first)

→ If tie FCFS ✓

* → SRTF → large CPU burst → low priority.

Preemption

Non preemption

preempt the CPU if priority of new arrived process is higher.

Q)

PID	BT	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	4
P ₄	1	5
P ₅	5	2

Grant's chart

0	1	6	16	17	19
P ₂	P ₅	P ₁	P ₃	P ₄	

W(P₁) = 6

W(P₂) = 0

W(P₃) = 16

W(P₄) = 18

W(P₅) = 1

avg = $\frac{41}{5} = 8.2ms$

problem :- Starvation

- ready to run → blocked ✓
- can leave some low priority process waiting.

→ Solved by Aging

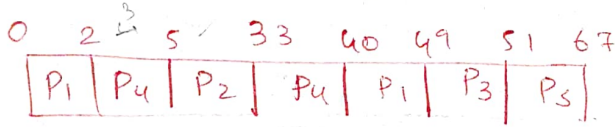
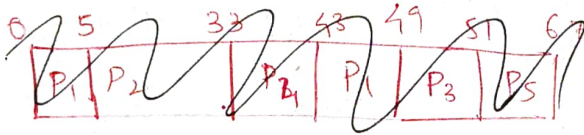
increase priority of process that wait for long time.

Q)

PID	AT	BT	Priority
P ₁	0	11	2
P ₂	5	28	0
P ₃	12	2	3
P ₄	2	10	1
P ₅	9	16	1

AWT ??

Preemptive



$$W(P_1) = 40 - 2 - 0 = 38$$

$$W(P_2) = 5 - 5 = 0$$

$$W(P_3) = 49 - 12 = 37$$

$$W(P_4) = 33 - 3 - 2 = 28$$

$$W(P_5) = 51 - 9 = 42$$

$$AWT = 29ms$$

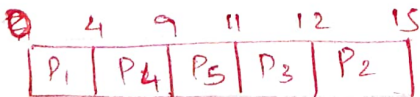
Q)

PID	AT	BT	Priority
P ₁	0	4	2
P ₂	1	3	3
P ₃	2	1	4
P ₄	3	5	5
P ₅	4	2	5

high → high priority

Non-preemptive

AW, ATOT



$$AWT(P_1) = 0 - 0 = 0$$

$$AWT(P_2) = 12 - 1 = 11$$

$$AWT(P_3) = 11 - 2 = 9$$

$$AWT(P_4) = 4 - 3 = 1$$

$$AWT(P_5) = 9 - 4 = 5$$

$$TOT(P_1) = 4 - 0 = 4$$

$$TOT(P_2) = 15 - 1 = 14$$

$$TOT(P_3) = 12 - 2 = 10$$

$$TOT(P_4) = 9 - 3 = 6$$

$$TOT(P_5) = 11 - 4 = 7$$

$$8.2$$

$$8.2$$

5) Round Robin Scheduling

- Time sharing System.
- FCFS with preemption added
- Time Quantum (Time Slice) allocating CPU to each process for a time of interval of the

Ready Queue → Circular

- Ready Queue → FIFO Queue
- New process → Tail
- First process to interrupt after 1 Time Quantum & dispatcher

CPU time < 1TQ

↓
Removes voluntarily
↓
Next process

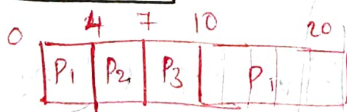
CPU time > 1TQ,
Cause interrupt to OS

↓
Context switch &
Process put @ tail of queue
↓
next process

Q1

PID	BT
P ₁	24
P ₂	3
P ₃	3

Time Quantum = 4ms



M1:-

$$TAT = CT - AT$$

$$WT = TAT - BT$$

M2:-

$$WT = (\text{Last start time}) - (\text{Arrival time}) - (\text{preemption} \times T.Q)$$

$$WT(P_1) = 26 - 0 - (5 \times 4) = 6$$

$$WT(P_2) = 4 - 0 - (0 \times 4) = 4$$

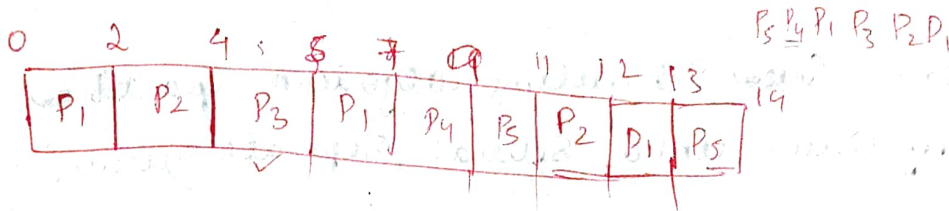
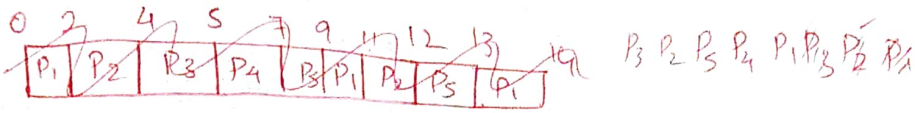
$$WT(P_3) = 7 - 0 - 0 = 7 \checkmark$$

Q)

PID	AT	BT
P ₁	0	5 → 3 → 1
P ₂	1	3 → 1
P ₃	2	1 ×
P ₄	3	2 ×
P ₅	4	3 → 1

RR
TQ = 2 units

~~✗~~
DO

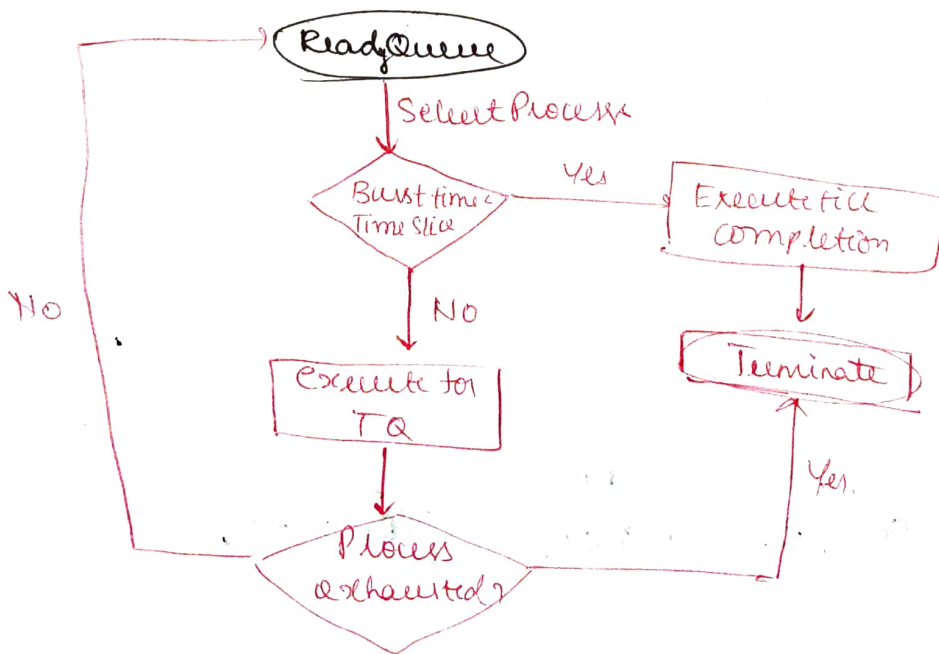


⇒ Write the ready Que & clock in background

TAT = completion time - Arrival

WT = TAT - BT

TAT(P ₁) = 13 - 0 = 13	WT(P ₁) = 13 - 5 = 8
TAT(P ₂) = 12 - 1 = 11	WT(P ₂) = 11 - 3 = 8
TAT(P ₃) = 5 - 2 = 3	WT(P ₃) = 3 - 1 = 2
TAT(P ₄) = 9 - 3 = 6	WT(P ₄) = 6 - 2 = 4
TAT(P ₅) = 14 - 4 = 10	WT(P ₅) = 10 - 3 = 7
8.6	5.8



Overhead (δ)

$$\eta = \frac{q}{q + \delta}$$

TQ = q ✓
 ↓
 TO = q + δ ✓

6) Multilevel Queue Scheduling.

- class of scheduling algorithm are classified into different group.
- Different response time requirements
- foreground process may have priority.

A Multilevel Queue Scheduling algorithm partitions the ready Queue into several separate queues.

(each process are permanently assigned to one queue (each has own scheduling alg).)

Ex:- foreground process (RR) | background process (FCFS)

⊗ There Must be Scheduling among the Queues.
[fixed-priority preemptive Scheduling]

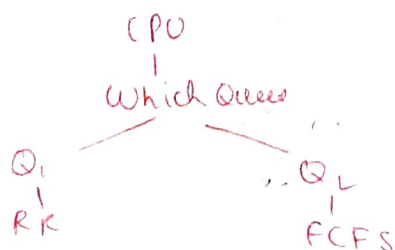
Ex:- High

- System process →
- Interactive process →
- Batch process →
- Student process →

Low

⊗ Processes Can't be Moved between Queues.

RR &
Ex:- $Q_1, Q_2 \rightarrow FCFS$ $P(Q_1) > P(Q_2)$



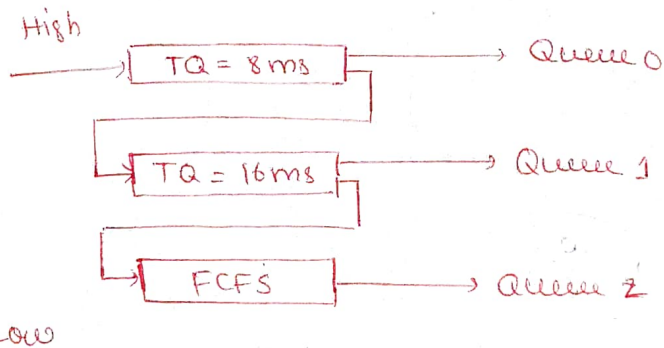
- Ready Queue → Multiple Separate Queues
- each queue has own alg.
- Switching B/W Queue is performed acc to priority.

Multilevel Feedback - Queue Scheduling

- ↳ allows to move b/w queues.
- ↳ separate queues acc. to CPU burst.
- ↳ Too Much CPU time $\rightarrow$ moved to lower-priority queue & vice versa.
- ↳ I/O bound & interactive } Higher priority

THIS FORM OF AGING PREVENTS STARVATION

Ex:-



Parameters:-

- $\rightarrow$ No. of Queues
- $\rightarrow$ Each Queue Scheduling
- $\rightarrow$ method used when upgrading to higher queue
- $\rightarrow$ method used when degrading to lower queue
- $\rightarrow$ To which queue should it move

Highest response ratio Next (HRRN) CPU Scheduling:-

- $\rightarrow$ CPU assigned to process having highest response ratio
- $\rightarrow$ Tie $\equiv$ FCFS
- $\rightarrow$ only non-preemptive

$$RR = \frac{W+B}{B}$$

$W \rightarrow$ Waiting time so far
 $B \rightarrow$ Burst time.

$$\frac{TAT}{BT}$$

$$\frac{TAT}{BT}$$

o Better than SJF Scheduling (limits waiting time)

o Burst time not known in advance.

Q1)

PID	AT	BT
P ₀	0	3
P ₁	2	6
P ₂	4	4
P ₃	6	5
P ₄	8	2

at 0 :- only P₀ (3)

at 3 :- only P₁ (9)

at 9 :- P₂, P₃, P₄

0	3	9	13	15	20
P ₀	P ₁	P ₂	P ₄	P ₃	

$$RR(P_2) = \frac{(9-4)+4}{4} = 2.25 \checkmark$$

$$RR(P_3) = \frac{(9-6)+5}{5} = 1.6$$

$$RR(P_4) = \frac{(9-8)+2}{2} = 1.5$$

$$= \frac{(13-6)+5}{5} = 2.4$$

$$= \frac{(13-8)+2}{2} = 3.5$$

$$1 + \frac{w}{B}$$

$$T_{AT} = w + B$$

$$w = T_{AT} - B$$

$$CT - AT$$

IPC (Inter process communication)

Independent :- not affected by others
Cooperating :- affected / affected.

IPC :-

- 1) Shared Memory.
- 2) Message Passing.

1) Shared

POSIX

① `<sys/mman.h>`
`<fcntl.h>`

② Path string
"/myshm"

③ visible @ /dev/shm

④ `int fd`
(file descriptor)

⑤ Mapping :- `mmap()`

Sys-V

① `<sys/ipc.h>`
`<sys/shm.h>`

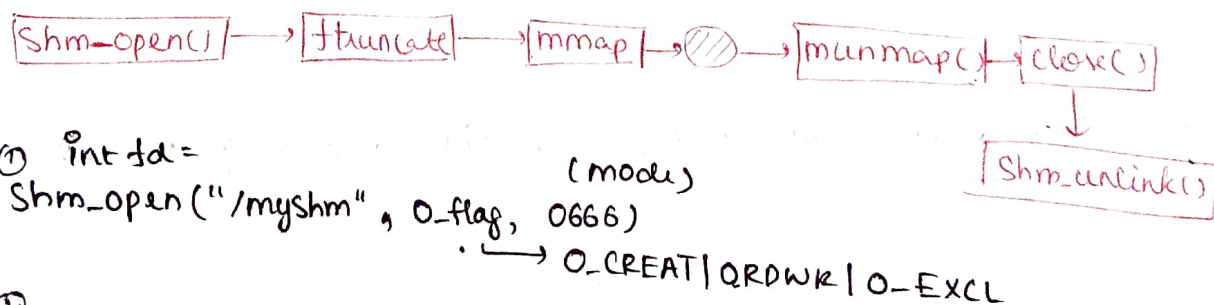
② Numeric key
(Key-t = `ftok("-", "-")`)

③ `ipcs -m`

④ `int shmid`

⑤ `Shmat`

POSIX :-



② `ftruncate(fd, size)` — **SIGBUS Error** ?? process attempts to write beyond limit.

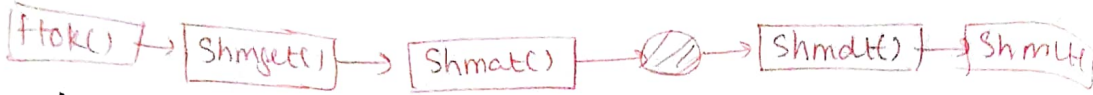
③ `int *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)`

④ `munmap(ptr, size)`

⑤ `close(fd)`

⑥ `shm_unlink("/myshm")`

System-V



- ① int key =
ftok = ("myfile", 'A')
- ② int shmid =
shmget (Key, size, IPC_CREAT|0666);
- ③ int *ptr =
shmat (shmid, NULL, 0)
 ↳ SHM-RDONLY
- ④ shmdt (ptr)
- ⑤ shmctl (shmid, IPC_RMID, NULL)

Pipe:-

- Send one process data to another.
- Kernel Buffer.

fd[2] ✓ [fd[0] - read ; fd[1] - write]

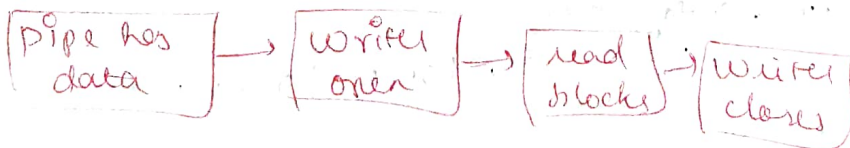
pipe (fd)

read (fd[0], buffer, n) ; close (fd[1])

write (fd[1], buffer, n) ; close (fd[0])

Unused pipe ends Must be closed! → A reader detects EOF only after all fd to write end are closed.
→ It will be waiting for More data instead of EOF

→ If writer didn't write anything, the reader just wait.



↓
read == 0 → EOF

SIGPIPE:- when reader is closed, writer keeps writing.

Synchronization

→ cooperating process that can affect / affected by other.

race condition:- situation where several process access & Manipulate same data

→ outcome depends on order.

→ Synchronization

Situation where Multiple processes access shared data & final result depends on order.

Critical Section:- part of program that access / modifies shared data (race).

✱

✱ Solution:-

- Mutual exclusion (only one process)
- Progress (a process that enters when no one is there should get fast)
- No starvation (bounded).

Note:-

→ ready & waiting for lock, don't keep getting denied.

✱ each process requests permission to critical section

✱ This is entry section

✱ Followed by exit section

✱ remaining code is remainder section



Locks & Unlocks

"How to ensure Mutual exclusion!
(Only one process enters critical)

lock() →  → unlock()

Bounded waiting:- Once a process requests to enter, there must be limit on how many times others can enter before it gets turn.

How to Lock?

A) INTERRUPTS

- Disable interrupts while executing critical section
- Simple: - Interrupts are disabled - context switching won't happen
- require privileges! - User can't disable interrupts
- Not suited for Multicore! - another system may access

B) SOFTWARE SOLUTION = Turn Variable.

Shared
int turn = 1;

P1

```
while(1){  
    while(turn == 2); (lock)  
    Critical  
    turn = 2; (unlock)  
}
```

P2

```
while(1){  
    while(turn == 1); (lock)  
    Critical  
    turn = 1; (unlock)  
}
```

- Mutual exclusion ✓
- Busy waiting
- needs alternate execution ***

C) SOFTWARE SOLUTION (ATTEMPT 2)

Shared

P2inside = false
P1inside = false

P1

```
while(1){  
    while(P2inside == true);  
    P1inside = true  
    Critical  
    P1inside = false  
}
```

P2

```
while(1){  
    while(P1inside == true);  
    P2inside = true;  
    Critical execution  
    P2inside = false  
}
```

- Needn't alternate execution in critical section ???
- doesn't guarantee Mutual exclusion

⇒ The Flag (P1inside, P2inside) is set after we break from the while loop.

D) SOFTWARE SOLUTION (ATTEMPT-3)

Shared

P2 wants to enter
P1 wants to enter

P1

```
while(1) {
    P1 wants to enter = True;
    while (P2 wants to enter);
    critical
    P1 wants to enter = false;
}
```

P2

```
while(1) {
    P2 wants to enter = True;
    while (P1 wants to enter);
    critical
    P2 wants to enter = false;
}
```

⇒ Achieves Mutual exclusion.

⇒ Doesn't Achieve progress (DEADLOCK)

(when both want to enter simultaneously both wait for both to become false)

PETERSON'S SOLUTION

Shared

P2 wants to enter
P1 wants to enter
favoured

flag[0]
flag[1]
→ Twin

P1

```
while(1) {
    P1 wants to enter = True;
    favoured = 2;
    while (P2 wants to enter &&
           favoured == 2);
    critical
    P1 wants to enter = false;
}
```

P2

```
while(1) {
    P2 wants to enter = True;
    favoured = 1;
    while (P1 wants to enter &&
           favoured == 1);
    critical
    P2 wants to enter = false;
}
```

favoured = 2 → If P2 also wants to enter, favor it

⚡ Whichever sets the favoured last loses

→ Achieves Mutual exclusion

→ DEADLOCK BROKEN.

BAKERY Algorithm

- Synchronization between $N > 2$ Processes.
- each has a token, whoever has smallest goes.

a) Simplified bakery algorithm :-

- $num[N] = \{0, 0, 0, \dots, 0\}$
- $num[i] \rightarrow$ the ticket number held by P_i
- $\rightarrow num[i] = 0 \rightarrow$ the process not trying to enter critical.

if some process has to enter:-

$$P_i = num[i] = \max(num[0], num[1], \dots, num[N-1]) + 1$$

The rule for entering is

P_i can enter only when every process has 0 or has a number greater than $num[i]$

Alg:-

lock(i) {

$num[i] = \max[num[0], num[1], \dots, num[N-1]] + 1;$

for ($p = 0$ to N) {

while ($num[p] \neq 0$ && $num[p] < num[i]$);

CRITICAL

unlock(i) {

$num[i] = 0$

}

- If two process does $num[i] = \max[_] + 1$, simultaneously then it will give same ticket. **DOORWAY**

b) Original bakery algorithm

- introduces choosing[N]
- initially choosing[i] = False.
- when taking ticket, tells everyone ✓

Alg:-

lock(i) {

 choosing[i] = True

 num[i] = MAX(num[j]) + 1 } — DOORWAY

 choosing[i] = False.

 for (p = 0 to N) {

 while (choosing[p])

 while (num[p] != 0 AND (num[p], p) < (num[i], i))

 }

critical

unlock(i) {

 num[i] = 0

}

$$(a, b) < (c, d) \equiv (a < c) \text{ or } [(a == c) \text{ And } (b < d)]$$

(X)

Sol ⁿ	Idea	Processes	Mutual exclusion	Progress	Bounded waiting	Problem
Disable Interrupt	"	Single CPU	✓	✓	✓	Privilege, Not Multicore
Attempt 1	turn variable	2	✓	X	X	may wait when other not interested
Attempt 2	2 flags	2	X	—	X	Both enter simultaneously
Attempt 3	announce before lock	2	✓	X	X	both may wait
Peterson	2 Flags + Favourite	2	✓	✓	✓	Busy wait
Bakery	Tickets	N	✓	✓	✓	Busy wait

Both are busy waiting (spinning)

↳ continuously checking in a tight loop until it becomes True.

Active waiting: - Thread remains active without sleeping because

→ Avoids overhead of sleeping then waking

draw

→ High CPU waste.

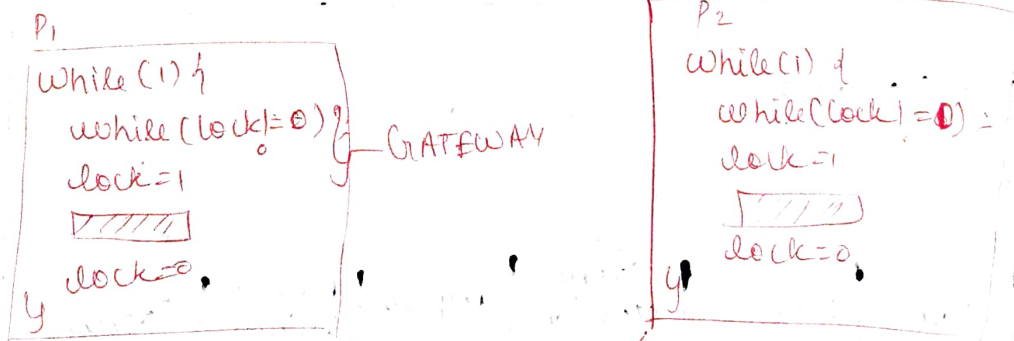
Use for → Short waits
→ Multi processor
→ low-level programming

Avoid for → long waits (Mutexes / Semaphores) to Make thread sleep
→ Single Core - System.

Implement locking in Hardware

Test & Set Instruction:-

lock = 0



Make Gateway atomic

```
int test_and_set(int *L) {  
    int *prev = L;  
    *L = 1;  
    return prev;  
}
```

write to memory location & return old value

```

while(1) {
    while (test_and_set(&lock) == 1);
    // Critical Section
    lock = 0;
}

```

→ if CPU's execute this @ Same time, the hardware ensures that one test-and-set does before other.

xchg → atomically exchanges register with memory value.

High Level Constructs:-

- Spinlock
- Mutex
- Semaphore.

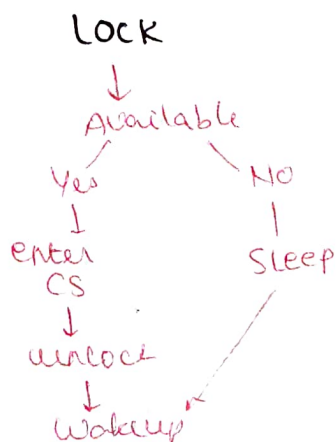
a) Spinlock:- (Loop)

- One process will acquire lock
- Other will wait in loop repeatedly checking if lock available.

→ Some process may complete before going to sleep & later waking up.

b) Mutex:-

- If critical section is locked then SLEEP
- while unblocking, wakeup.



```

void unlock(int *L) {
    L = 0;
    wakeup();
}

```

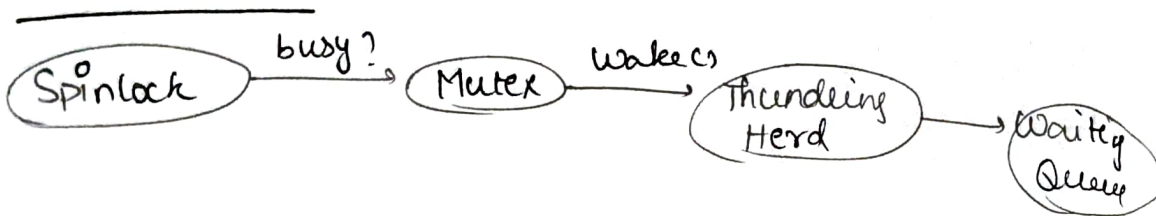
```

void lock(int *L) {
    while(1) {
        if (xchg(L, 1) == 0)
            break;
        else
            sleep();
    }
}

```

Thundering Herd Problem:-

- A large number of process may wakeup simultaneously when the event occurs.
- Leads to several context switching
- All go back to Sleep except desired one
 - Cause Starvation



- ⇒ When entering critical section, push into queue. when exiting, wake up only the first process in queue.

DEADLOCKS

↳ Situation where program runs indefinitely without making any progress.

↳ Each program waits for event caused by another

1) Process:- fork()

exec()

wait()

Zombie

Orphan

2) Scheduling:- Scheduler types

Long / Medium / Short term

Queues

Context Switching

FCFS, SJF, SR, TF, Round Robin

Throughput

Ideal time

HRRN

Multilevel Queue

MLFQ (Multilevel Feedback Queue)

Avg WT, TAT, Response Time.

3) IPC :- Pipe

Shared Memory

Shared Memory in Linux

4) Synchronization:- Race Condition

Critical Section

Mutual Exclusion

Progress

Bounded waiting / no Starvation

Lock / Unlock

Disable

Software Attempts 1, 2, 3

Peterson Algorithm

Simplified Bakery Algorithm

Original Bakery Algorithm

Test-and-Set

Spinlock

Mutex

Thundering Herd

Notes:-

Race condition: outcome of concurrent execution depends on particular order.

count++;

P1: LOAD COUNT $\rightarrow$ 5

P2: LOAD COUNT $\rightarrow$ 5

P1: ADD $\rightarrow$ 6

P2: ADD $\rightarrow$ 6

P1: STORE $\rightarrow$ 6

P2: STORE $\rightarrow$ 6

In Multicore CPU

Build memory fences to make atomic

Peterson works P0 & P1 request same time.

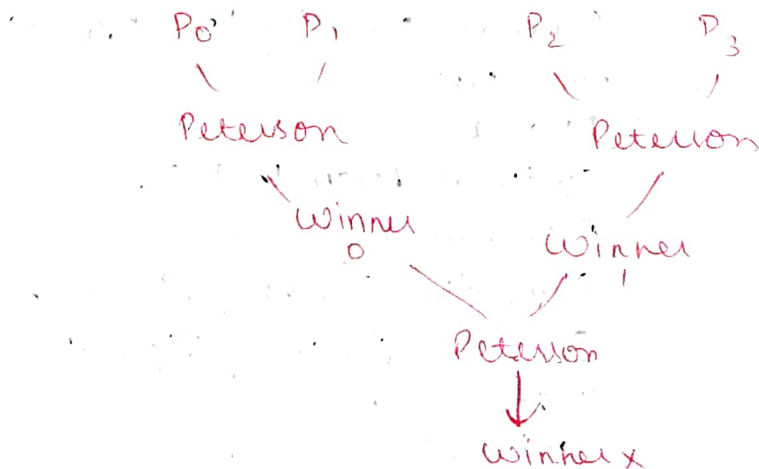
$\rightarrow$ flag[i] = True for both, if second survives
Turn = 0

$\rightarrow$ P0:- while (flag[0] && turn == 1); $\rightarrow$ P0 enters

$\rightarrow$ P1:- while (flag[0] && turn == 0); waits

$\Rightarrow$ if flag[0] or flag[1] is not interested, then the other can enter.

For N Process:- Tournament Tree of pairwise.
 $O(\log n)$



Bakery

```
bool choosing[n] = {false};
```

```
int number[n] = {0};
```

```
choosing[i] = true;
```

```
number[i] = max(number[0...n-1]) + 1
```

```
choosing[i] = false;
```

```
for (int j = 0 to n-1)
```

```
while (choosing[j])
```

```
while (num[j] != 0 && (num[j], j) < (num[i], i))
```

```
num[i] = 0
```

```
num[i] = 0
```

Proof by contradiction - Mutual exclusion in bakery..

→ P_i & P_j are in CS (P_i has smallest ticket)

→ for P_j to enter, every $k \rightarrow$ false ($k=i$)

$num[i] == 0$ or $(num[i], i) < (num[j], j)$

→ sent P_i in CS, so not 0. & assumed.

→ false ✓

Bakery: Ticket Queue

$O(n)$ per entry/exit

Weak memory order (memory fence needed)

Test & Set

```
bool testandset (bool *lock)
```

```
bool prev = lock
```

```
*lock = True
```

```
return prev
```

```
bool L = false
```

```
while (testandset(&L))
```

```
lock = false
```

```
lock = false;
```

→ Single, atomic, no interrupt can modify lock

→ return old value

Swap to implement lock

```
void swap (bool *a, bool *b)
{
    bool temp = *a;
    *a = *b;
    *b = temp;
}
```

```
bool L = False;
bool key = True; // (purpose)
do {
    swap(&L, &key);
} while (key == true);
//
L = False;
```

TestandSet + auxiliary array to overcome bounded waiting

```
bool lock = False;
bool waiting[n] = {False};
// lock i
waiting[i] = true;
while (waiting[i] && testandtest(&lock));
waiting[i] = false;
```

// exit

int j = (i+1) % n

while (j != i && !waiting[j])

j = (j+1) % n

if (j == i) lock = False

else waiting[j] = false

Implement spinlock using test & set.

```
typedef struct {
```

```
    bool locked;
```

```
} spinlock_t
```

```
void acquire (spinlock_t *lock)
```

```
    while (TestandSet(&lock->locked));
```

```
}
```

```
void release (spinlock_t *lock) {
```

```
    lock->locked = false;
```

```
}
```